

Ejercicio 2: Optimización de Transporte en Almacenes (Programación Dinámica)

Enunciado

Una empresa de distribución de un determinado producto tiene dos almacenes A_1 y A_2 desde los cuales satisface los pedidos de sus clientes C_j , $1 \leq j \leq n$. Denotamos por d_j el número de unidades de la demanda del cliente C_j , $1 \leq j \leq n$, y por r_i el número de unidades de inventario en A_i , $1 \leq i \leq 2$. Se supone que $r_1 + r_2 = d_1 + \dots + d_n$. Sea $c_{ij}(x_{ij})$ el coste de transportar x_{ij} unidades del producto desde el almacén A_i hasta el cliente C_j . El problema del almacén consiste en calcular los enteros no negativos x_{ij} , $1 \leq i \leq 2$, $1 \leq j \leq n$, tales que $x_{1j} + x_{2j} = d_j$, $1 \leq j \leq n$ y que $\sum_{i,j} c_{ij}(x_{ij})$ sea mínimo.

Sea $g_i(x)$ el coste incurrido por una solución óptima del problema de satisfacer los pedidos de los clientes $C_1, \dots, C_i$ suponiendo que A_1 tiene un inventario de x unidades (y, por tanto, el inventario de A_2 es $\sum_{1 \leq j \leq i} d_j - x$). Entonces, el coste de una solución óptima del problema del almacén es $g_n(r_1)$.

Utilizar el principio de optimalidad para encontrar una relación recurrente para los $g_i(x)$. Diseñar un algoritmo eficiente para resolver esa relación recurrente y obtener los valores óptimos de x_{ij} , $1 \leq i \leq 2$, $1 \leq j \leq n$.

1. Aplicación del Principio de Optimalidad

Para satisfacer la demanda de los clientes de forma secuencial, planteamos un enfoque por etapas de decisión donde cada etapa i representa la asignación del pedido del cliente C_i :

1. En la etapa i , debemos tomar la decisión de enviar x_{1i} unidades desde el almacén A_1 al cliente C_i .
2. Dado que el cliente C_i requiere exactamente d_i unidades, el resto de su demanda debe satisfacerse obligatoriamente desde el almacén A_2 :

$$x_{2i} = d_i - x_{1i}$$

3. Para que la decisión sea válida, la cantidad enviada de A_1 al cliente C_i debe cumplir:
 - No ser negativa ($x_{1i} \geq 0$).
 - No superar la demanda del cliente ($x_{1i} \leq d_i$).
 - No superar el inventario actual asignado a A_1 en esta etapa ($x_{1i} \leq x$).
 - De manera análoga, para A_2 , no podemos enviar más unidades de las requeridas ni más de las que tiene su inventario: $x_{2i} \geq 0 \implies x_{1i} \leq d_i$, lo cual ya está cubierto.

Justificación por el Principio de Optimalidad de Bellman

decisiones iniciales, las decisiones restantes deben constituir una política óptima con respecto al estado resultante de la primera decisión."

Si decidimos enviar x_{1i} unidades desde A_1 al cliente C_i , incurrimos en un coste inmediato de $c_{1i}(x_{1i}) + c_{2i}(d_i - x_{1i})$. El inventario remanente en el almacén A_1 para satisfacer a los primeros $i - 1$ clientes será exactamente de:

$$x' = x - x_{1i}$$

Por tanto, el coste total de nuestra decisión es la suma del coste del cliente C_i y el coste óptimo para los primeros $i - 1$ clientes con el inventario ajustado $g_{i-1}(x - x_{1i})$.

Relación de Recurrencia de la Función Óptima

La ecuación recursiva para $g_i(x)$ se define como:

$$g_i(x) = \min_{0 \leq x_{1i} \leq \min(x, d_i)} \left\{ c_{1i}(x_{1i}) + c_{2i}(d_i - x_{1i}) + g_{i-1}(x - x_{1i}) \right\}$$

Límites del Estado x :

El inventario x de A_1 en la etapa i puede tomar cualquier valor entero que respete los límites físicos:

- $0 \leq x \leq \min \left(r_1, \sum_{j=1}^i d_j \right)$

Casos Base (Etapa 0):

- $g_0(0) = 0$
- $g_0(x) = \infty$ para todo $x > 0$ (representa un estado de inventario inconsistente).

2. Algoritmo de Programación Dinámica (Bottom-Up)

Para implementar esta recurrencia de forma eficiente, definimos una tabla bidimensional $DP[i][x]$ de dimensiones $(n + 1) \times (r_1 + 1)$. Para poder reconstruir la solución óptima (los valores exactos de x_{ij}), utilizaremos una matriz auxiliar `decisión[i][x]` que almacenará la cantidad x_{1i} elegida que minimiza la ecuación en cada estado.

Para optimizar el tamaño de las búsquedas en el bucle, calculamos los sumatorios acumulados de la demanda $S_i = \sum_{j=1}^i d_j$.

Pseudocódigo del Algoritmo

```

Algoritmo OptimizacionAlmacen(n, d, r1, c1, c2)
    // Entrada:
    //   - n: Número de clientes
    //   - d[1..n]: Vector de demandas de los clientes
    //   - r1: Inventario disponible en el almacén A1
    //   - c1[1..2][1..n](cant): Matrices de coste de transporte para A1 y A2
    // Salida:
    //   - Coste mínimo y matriz de transporte x[1..2][1..n] óptimo

```

```

S[0] ← 0
Para i ← 1 hasta n hacer
    S[i] ← S[i-1] + d[i]
FinPara

// 2. Crear tablas de Programación Dinámica
Crear matriz DP[0..n][0..r1]
Crear matriz decision[1..n][0..r1]

// Inicializar caso base (i = 0)
DP[0][0] ← 0
Para x ← 1 hasta r1 hacer
    DP[0][x] ← INFINITO
FinPara

// 3. Rellenar la tabla iterativamente
Para i ← 1 hasta n hacer
    // El rango de inventario x viable para A1 está limitado por r1 y la demanda
    limite_x ← Minimo(r1, S[i])

    Para x ← 0 hasta limite_x hacer
        DP[i][x] ← INFINITO

        // Probar todas las decisiones posibles de envío desde A1 para el cliente i
        limite_envio ← Minimo(x, d[i])
        Para x1i ← 0 hasta limite_envio hacer
            // El inventario remanente en A1 debe ser coherente con la etapa anterior
            Si (x - x1i ≤ S[i-1]) entonces
                coste_actual ← c1[i][x1i] + c2[i][d[i] - x1i] + DP[i-1][x - x1i]

                Si (coste_actual < DP[i][x]) entonces
                    DP[i][x] ← coste_actual
                    decision[i][x] ← x1i // Guardar la decisión óptima
            FinSi
        FinSi
    FinPara
FinPara

// 4. Reconstrucción del camino óptimo
Crear matriz resultado_x[1..2][1..n]
inventario_restante ← r1

Para i ← n decreciendo hasta 1 hacer
    x1i_optimo ← decision[i][inventario_restante]

    resultado_x[1][i] ← x1i_optimo // Unidades enviadas desde A1
    resultado_x[2][i] ← d[i] - x1i_optimo // Unidades enviadas desde A2

    // El inventario se reduce por lo que acabamos de enviar desde A1
    inventario_restante ← inventario_restante - x1i_optimo
FinPara

Retornar DP[n][r1], resultado_x
FinAlgoritmo

```

3. Análisis de Complejidad

- **Llenado de la matriz:** El algoritmo consta de tres bucles anidados. El bucle exterior recorre las n etapas. El segundo bucle recorre los posibles estados de inventario x (de 0 a r_1). El tercer bucle interno prueba las opciones de envío de la demanda $d[i]$ (desde 0 hasta d_i).
- Por lo tanto, el número total de operaciones de evaluación está acotado por:

$$\mathcal{O} \left(n \cdot r_1 \cdot \max_{1 \leq i \leq n} d_i \right)$$

Si el tamaño máximo de demanda d_i está acotado por una constante relativamente pequeña, la complejidad temporal se simplifica a $\mathcal{O}(n \cdot r_1)$, lo cual es pseudo-polinomial y sumamente rápido en comparación con una búsqueda de fuerza bruta exponencial $\mathcal{O}(\prod d_i)$.

Complejidad Espacial

- **Espacio Auxiliar:** Necesitamos las tablas `DP` y `decision` de tamaño $(n + 1) \times (r_1 + 1)$.
- Por tanto, la complejidad espacial es de $\mathcal{O}(n \cdot r_1)$, lo cual resulta totalmente viable para valores estándar de inventario y número de clientes en sistemas de logística y distribución reales.